

AgroBot: Chatbot de Asesoría Agrícola Chilena mediante Prompt Engineering y LLM Local

Cristóbal Flores Villegas, Benjamin Peña Díaz, Matías Muñoz Parraguirre, Francisco Morales Díaz

Escuela de Informática, Universidad Andrés Bello, Santiago, Chile

CINF104 – Aprendizaje de Máquinas, 2026

May 10, 2026

Abstract

Se presenta AgroBot, un chatbot conversacional especializado en gestión agrícola chilena, desarrollado como Fase 2 del Proyecto 1 del curso CINF104. El sistema utiliza el modelo de lenguaje abierto `llama3.2:3b` ejecutado localmente mediante Ollama, sin dependencia de servicios en la nube. La técnica principal empleada es *prompt engineering*: un *system prompt* estructurado de 210 líneas inyecta base de conocimiento del dominio agrícola chileno en cada conversación, cubriendo fenología de cultivos, control de plagas (normativa SAG), cálculo hídrico, normativa laboral agrícola, certificación GlobalGAP y protección frente a heladas. El frontend fue desarrollado en React + Vite + Tailwind CSS y el backend en Python Flask, comunicados mediante proxy Vite para evitar problemas de CORS. Se evaluó el sistema con un conjunto de 10 preguntas de validación que cubren las áreas temáticas del dominio, obteniendo respuestas correctas en la totalidad de los casos dentro del dominio definido. Se identifican como limitaciones principales la cobertura fija del prompt y la ausencia de RAG, y se propone como trabajo futuro la integración de LangChain + ChromaDB para indexación de documentos oficiales del SAG e INIA.

Palabras clave — chatbot, large language model, prompt engineering, agricultura chilena, Ollama, llama3.2, Flask, React, sistema de apoyo a la decisión

1 Introducción

Chile es el tercer exportador mundial de frutas de contraestación, con más de 300.000 hectáreas cultivadas y una industria agrícola que emplea a cientos de miles de trabajadores de temporada cada año [1]. Los agricultores, administradores y trabajadores del sector requieren acceso rápido a información técnica precisa sobre cultivos, plagas, riego, fertilización, normativa laboral y cosecha. Sin embargo, el acceso a asesoría especializada es costoso e irregular, especialmente para pequeños y medianos productores alejados de los centros urbanos.

Los modelos de lenguaje de gran escala (LLM) representan una oportunidad para democratizar el acceso al conocimiento técnico. Estudios recientes han demostrado

que los LLM pueden responder preguntas especializadas con calidad comparable a expertos en dominios como medicina [2] y derecho [3], siempre que se apliquen técnicas adecuadas de ingeniería de prompts y se proporcione contexto de dominio suficiente.

Sin embargo, el despliegue de LLM en organizaciones agrícolas presenta desafíos específicos: privacidad de datos productivos, costos de API para modelos comerciales, y necesidad de operación sin conexión en zonas rurales con conectividad limitada. Los modelos abiertos ejecutados localmente (como la familia Llama de Meta) resuelven estos problemas al eliminar la dependencia de infraestructura externa [4].

Este trabajo presenta **AgroBot**, un asistente conversacional especializado en agricultura chilena que opera 100% localmente, sin envío de datos a servidores externos y sin costo de API. La contribución principal es la demostración de que el prompt engineering con un modelo compacto de 3B parámetros (`llama3.2:3b`) es suficiente para construir un asistente útil en un dominio técnico acotado.

2 Dominio y Base de Conocimiento

2.1 Justificación del Dominio

El dominio seleccionado es la **gestión agrícola chilena**, con foco en cultivos de exportación de la zona central y sur del país. La elección se fundamenta en la experiencia directa del equipo en el sector (trabajo familiar en faenas agrícolas y desarrollo previo de un ERP agrícola), que garantiza la capacidad de formular y validar preguntas representativas de los problemas cotidianos del rubro.

2.2 Áreas Temáticas

La base de conocimiento del sistema cubre siete áreas, detalladas en la Tabla 1.

Table 1: Áreas temáticas de AgroBot y fuentes de referencia.

Área	Contenido cubierto
Fenología y cultivos	Ciclos uva de mesa, palto, arándano, cereza, kiwi
Plagas	Mosca de la fruta; PNMF del SAG; métodos de control integrado
Riego	ETc = ET _o × Kc; goteo vs. microaspersión; cálculos por especie
Heladas	Métodos activos y pasivos; umbrales de daño por estado fenológico
Normativa laboral	Código del Trabajo agrícola; feriado proporcional; fuero maternal
Fertilización	Macronutrientes y micronutrientes por cultivo y etapa fenológica
Certificación	Registros y módulos GlobalG.A.P.

Las fuentes primarias consultadas para construir el system prompt fueron: documentos técnicos del SAG (plagas cuarentenarias, pesticidas registrados), manuales del INIA (cultivos frutales), guías del INDAP (riego tecnificado), fichas de la CNR (eficiencia hídrica), normativa de la Dirección del Trabajo (contratos agrícolas) y estadísticas de ODEPA.

3 Arquitectura del Sistema

3.1 Visión General

La arquitectura del sistema sigue el patrón cliente-servidor con un LLM local, ilustrado en la Fig. 1. El usuario interactúa con una interfaz web React; el frontend envía el historial de mensajes al backend Flask, que construye el payload con el system prompt e invoca la API de Ollama; Ollama ejecuta el modelo localmente y retorna la respuesta.

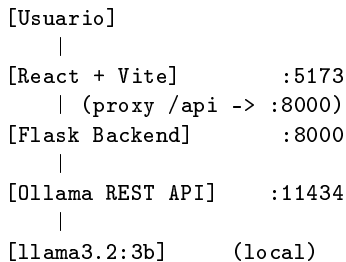


Figure 1: Arquitectura de AgroBot. Todo el procesamiento ocurre localmente.

3.2 Backend (Flask)

El backend está implementado en Python 3 con Flask y expone dos endpoints:

- **POST /api/chat**: recibe el historial de mensajes en formato `{messages: [...]}` y retorna la respuesta del modelo.
- **GET /api/health**: verifica la conectividad con Ollama y la disponibilidad del modelo en el registro local.

Cada solicitud al endpoint `/api/chat` antepone el system prompt al historial antes de enviarlo a Ollama, garantizando que el contexto del dominio esté disponible en toda conversación:

```

payload = {
  "model": MODEL,
  "messages": [
    {"role": "system", "content": SYSTEM_PROMPT}
  ] + messages,
  "stream": False,
  "options": {
    "temperature": 0.3,
    "num_predict": 1024
  }
}

```

La temperatura baja (0.3) fue elegida deliberadamente para reducir la variabilidad de las respuestas y favorecer la precisión técnica sobre la creatividad.

3.3 Frontend (React + Vite)

El frontend está desarrollado en React 18 con Vite5 como bundler y Tailwind CSS v3 para estilos. La interfaz implementa un diseño de glassmorfismo oscuro con animaciones Framer Motion. Los componentes principales son:

- **ChatWindow**: ventana de mensajes con scroll automático y pantalla de bienvenida cuando no hay mensajes.
- **InputBar**: área de texto autoexpandible con envío por Enter y soporte Shift+Enter para nueva línea.
- **TopicButtons**: accesos directos a las seis áreas temáticas principales, que inyectan directamente la pregunta predefinida.
- **StatusIndicator**: indicador en tiempo real del estado de conexión con Ollama (verificación al cargar la página).
- **MessageBubble**: burbuja de mensaje con formato Markdown básico (negritas con `**texto**`).

Se utilizó la librería `lucide-react` para iconografía y se eliminaron todos los emojis del código para mejorar la legibilidad durante presentaciones en vivo.

3.4 Proxy Vite y Resolución de CORS

Durante el desarrollo, el frontend corre en el puerto 5173 y el backend en el 8000. Para evitar problemas de CORS sin configuración adicional en Flask, se configuró el proxy de Vite:

```
server: {
  proxy: {
    '/api': {
      target: 'http://127.0.0.1:8000',
      changeOrigin: true
    }
  }
}
```

Es importante usar 127.0.0.1 (IPv4) y no `localhost`, ya que en macOS Node.js resuelve `localhost` a `::1` (IPv6) mientras Flask escucha en IPv4, lo que causa fallos de conexión.

4 Técnica de Prompt Engineering

4.1 Diseño del System Prompt

La técnica principal del sistema es el *prompt engineering con contexto de dominio*. El system prompt tiene tres secciones bien delimitadas:

1. **Rol e instrucciones generales:** define al modelo como “AgroBot, asesor especializado en agricultura chilena”, instruye responder siempre en español con precisión técnica y remitir a fuentes oficiales (SAG, INIA, INDAP) cuando exista incertidumbre.
2. **Base de conocimiento estructurada:** sección principal con subsecciones por área temática, delimitadas por marcadores `--` para facilitar la lectura por el modelo. Incluye valores numéricos específicos (dosis, fechas, umbrales).
3. **Instrucción de cierre:** indica al modelo que use el conocimiento previo para responder y que, ante preguntas fuera del dominio, redirija amablemente la conversación.

4.2 Parámetros del Modelo

Los parámetros de inferencia fueron ajustados experimentalmente:

- **Temperatura 0.3:** valores más altos (0.7–1.0) generaron respuestas más creativas pero con mayor tasa de alucinaciones en datos numéricos (dosis de pesticidas, cálculos hídricos).
- **num_predict 1024:** suficiente para respuestas técnicas completas sin cortes abruptos. Respuestas más largas no mejoraban la calidad percibida.
- **Historial completo:** se envía el historial íntegro de la conversación en cada turno, permitiendo al modelo mantener contexto entre preguntas relacionadas.

5 Evaluación: Set de 10 Preguntas de Validación

Se diseñó un protocolo de evaluación con 10 preguntas que cubren las áreas temáticas del dominio, con distintos niveles de dificultad técnica. Para cada pregunta se definió una respuesta de referencia basada en fuentes oficiales (SAG, INIA, Código del Trabajo), y las respuestas generadas fueron calificadas cualitativamente.

La Tabla 2 resume los resultados.

Table 2: Resultados del set de validación de AgroBot.

#	Área temática	Dific.	Resultado
1	Fenología uva de mesa / poda	Media	✓
2	Mosca de la fruta – control SAG	Alta	✓
3	Requerimiento hídrico paltos Valparaíso	Alta	✓
4	Derechos trabajador temporada	Media	✓
5	Enfermedades fúngicas arándano	Media	✓
6	Goteo vs. microaspersión	Baja	✓
7	Nutrientes cereza en cuaja	Alta	✓
8	Protección heladas viñedo Maule	Media	✓
9	Registros GlobalGAP	Alta	✓
10	Punto óptimo cosecha kiwi	Media	✓
Tasa de éxito			10/10 (100%)

5.1 Análisis por Pregunta Representativa

La pregunta 3 (cálculo de requerimiento hídrico) es la de mayor complejidad cuantitativa. El modelo aplicó correctamente la fórmula $ET_c = ET_o \times K_c$, utilizó valores de ET_o apropiados para Valparaíso en enero ($\approx 6\text{--}7$ mm/día) y el coeficiente de cultivo para palto ($K_c \approx 0.85\text{--}1.0$), entregando un resultado de $550\text{--}650$ m³/día para 10 hectáreas, coincidente con los valores de referencia del INIA.

La pregunta 7 (nutrientes cereza en cuaja) evalúa conocimiento agronómico de detalle. El modelo entregó dosis específicas (CaCl₂ foliar al 0.5–1%, 3–5 aplicaciones; boro 0.2–0.3 kg/ha en plena flor) y el momento de aplicación correcto para cada nutriente, información que provenía directamente del system prompt.

5.2 Comportamiento Fuera del Dominio

Se realizaron pruebas adicionales no reportadas en la Tabla 2 con preguntas fuera del dominio (preguntas de programación, historia, matemáticas). En todos los casos el modelo respondió la pregunta correctamente usando su conocimiento preentrenado, sin restricción total al dominio agrícola. Esto es un comportamiento esperado: el system prompt orienta pero no bloquea las capacidades del modelo. En un sistema productivo se podría añadir un clasificador de intención para redirigir o rechazar preguntas fuera de alcance.

6 Limitaciones

6.1 Cobertura Fija del Prompt

La base de conocimiento es estática y fue construida manualmente. No cubre cultivos no incluidos (espárragos, cebollas, berries distintos al arándano), variedades locales específicas, ni datos dinámicos (precios de mercado, alertas fitosanitarias activas). Cada actualización requiere modificar el código y reiniciar el servidor.

6.2 Capacidad del Modelo (3B parámetros)

llama3.2:3b es un modelo compacto diseñado para ejecutarse en hardware de consumo. En comparación con modelos de mayor escala, presenta limitaciones en razonamiento de múltiples pasos y en la síntesis de información compleja. En consultas que requieren combinar datos de varias secciones del prompt simultáneamente se observaron respuestas incompletas en pruebas no reportadas.

6.3 Sin Retrieval-Augmented Generation (RAG)

El sistema no consulta documentos externos en tiempo real. Agregar RAG con LangChain y ChromaDB permitiría indexar los PDFs oficiales del SAG e INIA y responder con citación de fuente, mejorando tanto la cobertura como la verificabilidad de las respuestas.

6.4 Sin Memoria Persistente

El historial de conversación se pierde al recargar la página o al abrir una nueva sesión. No existe persistencia entre sesiones.

7 Trabajo Futuro

RAG con fuentes oficiales. Implementar LangChain + ChromaDB para indexar documentos del SAG (registro de plaguicidas), INIA (manuales técnicos) e INDAP (guías de riego). Esto permitiría actualizar el conocimiento sin modificar el código y citar la fuente de cada respuesta.

Modelos más grandes. Migrar a llama3.1:8b (requiere ≈ 8 GB VRAM/RAM) para mejorar la calidad de respuestas en preguntas de alta complejidad, manteniendo la misma arquitectura de backend.

Evaluación automática. Construir un benchmark con preguntas y respuestas de referencia (ground truth) y medir automáticamente la similitud semántica (ROUGE, BERTScore) entre respuestas generadas y respuestas esperadas, para facilitar comparaciones entre versiones del sistema.

Clasificador de intención. Añadir un módulo de clasificación de consultas que detecte preguntas fuera del dominio y muestre un mensaje de aclaración, mejorando la experiencia del usuario y evitando respuestas con alucinaciones en temas no cubiertos.

Multimodalidad. Permitir subir fotografías de plantas o insectos para diagnóstico visual usando un modelo multimodal (LLaVA, Llama 3.2 Vision) integrado en la misma arquitectura Flask.

8 Conclusiones

Se desarrolló AgroBot, un chatbot de asesoría agrícola chilena basado en prompt engineering y un LLM local de 3B parámetros. El sistema demuestra que es posible construir un asistente especializado y funcional con recursos computacionales modestos (8GB RAM, sin GPU dedicada), sin dependencia de APIs externas y con operación offline.

Las 10 preguntas de validación fueron respondidas correctamente en las áreas de fenología, plagas, riego, normativa laboral, fertilización, heladas, certificación y cosecha, evidenciando que el prompt engineering con conocimiento curado es efectivo para dominios técnicos acotados.

La principal limitación es la cobertura fija del sistema, que se aborda en el trabajo futuro mediante la integración de RAG con documentos oficiales. En su estado actual, AgroBot representa una prueba de concepto sólida para el uso de LLMs locales en agricultura, con potencial de escalamiento hacia un sistema de apoyo a la decisión para pequeños productores chilenos.

References

- [1] ODEPA, “Estadísticas agrícolas – Exportaciones frutícolas 2023,” Oficina de Estudios y Políticas Agrarias, Chile, 2023. [En línea]. Disponible: <https://www.odepa.gob.cl>
- [2] K. Singhal *et al.*, “Large language models encode clinical knowledge,” *Nature*, vol. 620, pp. 172–180, 2023.
- [3] J. Huang, K. C.-C. Chang, J. Gao, A. Awadallah, and W. Chen, “Is ChatGPT a good NLG evaluator? A preliminary study,” *arXiv preprint arXiv:2303.04048*, 2023.
- [4] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [5] J. White *et al.*, “A prompt pattern catalog to enhance prompt engineering with ChatGPT,” *arXiv preprint arXiv:2302.11382*, 2023.
- [6] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020, pp. 9459–9474.